Claude Code for Accounting Research

Module 5 — Writing, Safety, and the Verification Capstone

John Barrios

Yale School of Management

Today's Arc

- **0:00–0:15** Mini-lecture: writing as thinking, the banal/meaningful line
- **0:15–0:40** Demos: editor-not-rewriter, clean-room referee, style skills, referee-DAG, response-letter audit
- **0:40–1:05** Mini-lecture: verification debt, sandboxes, permission rules and hooks, credential hygiene, paper trails
- **1:05–1:15** Demo: git as the operating system
- **1:15–1:25** Break
- **1:25–2:45** Capstone lab: harden a Module 3 or Module 4 pipeline
- **2:45–3:05** Debrief + exit ticket + Module 6 handout

Writing as Thinking

- If you can't get a sentence to work, you usually haven't worked out the argument behind it
- Give the model a vague prompt and it does the thinking for you. The paragraph reads fine, but you never worked out the argument
- **The test:** can you explain it out loud, without the document or the model in front of you?
- A rushed, vague prompt gets you the model's generic default

Common Failure

Mistaking fluency for having thought it through

A smooth paragraph does not mean the argument in it is yours. “This sounds right” is not the same as “I could defend this in a seminar.”

The Banal/Meaningful Line

Banal — delegate

- Variable-definition tables
- Data appendices
- Table notes, robustness headers
- Anything the data already determine

Meaningful — protect

- Identification argument
- Statement of contribution
- Limitations section
- Any sentence where you make the argument

You decide where the line falls. Write down one task you will always delegate and one you will never let an agent start.

Common Failure

Treating the line as obvious in advance

The hard cases are in the middle. A paragraph describing a robustness check can look routine but contain a judgment call. If a sentence makes you *decide* something rather than report it, it is on the meaningful side, however short it is.

Anti-Homogenization

- LLMs make weak writing competent, and they make distinctive writing generic
- Journal house styles (JAR vs. TAR) differ, which helps a little, but every paper in a journal still drifts toward the same voice
- The style skills today exist to push back against that drift. They are not shortcuts

Demo 1: Editor, Not Rewriter

Ask for comments, not rewrites. The exact prompt:

```
Please review this Identification section. I want
feedback in the style of a New York Times editor --
sharp on argument and clarity. Do not directly edit
my writing. Instead, insert inline <!-- comments -->
at the specific places where the argument is weak,
unclear, or could be tightened. Do not rewrite any
sentences yourself; leave the prose exactly as
written and let the comments carry the feedback.
```

Without “do not,” you get a rewrite back. With it, the model points to the weak spots and you do the rewriting yourself.

The Clean-Room Referee

- If you run the editor prompt in the session where you wrote the section, the model has heard all your justifications. It will tend to accept your design
- A real referee has heard none of them. Run the critique separately:
  - in a fresh session started in an empty folder that holds only the draft, so there is no CLAUDE.md or `DECISIONS.md` to read
  - with a deny rule on `Edit(./paper/**)`, so it cannot edit the paper. “Do not edit” in the prompt is only a request
  - writing its report to `review/`, so you can answer it point by point
- Criticize the paper, not the authors. Comments about the authors give you nothing to fix

Builds on Scott Cunningham’s Referee 2 persona (MixtapeTools) and the Editor persona in Alessandro Spina’s Claude Code materials.

Clean-Room Referee: the Prompt

```
You are refereeing this paper for the Journal of Accounting Research. You
have only the manuscript in paper/. Do not edit any file. Read the
identification section and the sample-construction section, then write a
report to review/referee_identification.md. For every sample filter the
paper states, check whether the attrition it implies is consistent with
the N reported in Table 1, and list any filter whose effect on the sample
is never reported. Aim the criticism at the paper, not the authors.
```

Accounting papers often describe one sample and build another. A reviewer who never saw your cleaning script has to rebuild the sample from the text, which is what a journal referee does.

Common Failure

A referee that agrees with you

Models tend to tell you what you want to hear. If the critique is mostly praise, or it backs down as soon as you object, it is not a referee report. Ask for the three most serious problems rather than an overall assessment. When you disagree, write your answer in the report file, not in the chat.

Demo 2: Style Skills as SOPs

- `barrios-voice`: built from my published papers. It gets real patterns right and sometimes overdoes them. It reflects how I wrote then, so it needs updating
- `econ-humanizer`: flags common AI phrasing, such as inflated claims of importance, stacked hedges, and “this paper contributes to the growing literature”
- `latex-tables`: handles the routine table formatting
- As in Module 3, a skill is a written procedure, not a new ability

Demo 3: Referee-DAG

- `strategic-revision` turns four referee reports into a task list
- It splits comments into tasks, works out which tasks depend on others, groups what can run in parallel, and flags the critical path and conflicting requests
- The habit matters more than the tool: break the revision into tasks, map what depends on what, and find conflicts before you start writing

Auditing the Response Letter

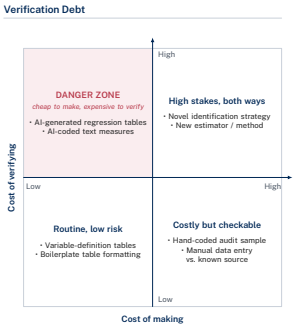
The DAG plans the revision. The response letter says it happened. An LLM will write "we now report…" whether or not anyone ran the analysis.

Claim in the letter	Where in the manuscript	Commit that implements it
"We add firm fixed effects to Table 4"	Table 4, cols. 3–4; §5.2	`a41c9e2`: adds `firm_fe` to the Table 4 script
"We now cluster at the industry level"	Table 4 notes	**(none found)**

- An empty commit cell is the dangerous case: the text was changed to match the letter, but the numbers were not
- Claude fills in the columns from the documents and `git log`, and leaves a cell empty rather than guessing. You check every empty cell

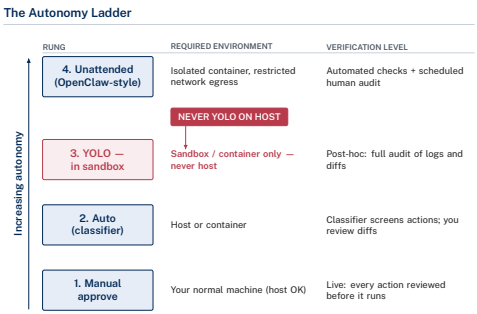
Adapted from the R&R mode of Spina's Editor persona; the commit column is this course's addition.

Verification Debt: the 2×2



The risky tasks are cheap to make and costly to verify. The more you let the agent run unsupervised, the faster unchecked work piles up. Exercise: place two of your own tasks from your Module 2–4 labs on the grid, with a partner if you have one (three minutes).

Environment First: the Autonomy Ladder



Skipping permissions inside a sandbox is a different risk from skipping them on your own machine, not a smaller version of it. Inside a sandbox, `cat ~/.ssh/id_ed25519` returns "Operation not permitted."

Installing a Skill Is Installing Software

- A skill is someone else's instructions running on your machine. Read it before you install it, as you would read a script before running it on your data
- A Markdown file can still tell an agent to do something you would not approve
- Trust a skill from Anthropic or your own group more than one from a little-known repo linked in a forum post

Common Failure

Treating sandboxing as a substitute for data governance

A container limits what an agent can *do*. It does not limit what the agent can *see* in a folder you mount, and it does not stop that folder's contents from being sent to the model. Mount only what the task needs.

Writing the Boundary Down

```
{
  "permissions": {
    "deny": [
      "Read(./.env)",
      "Read(./.env.*)",
      "Read(~/.ssh/**)",
      "Read(~/.wrds-mcp.env)",
      "Read(~/.claude.json)",
      "Edit(./data/raw/**)",
      "Bash(rm -rf *)"
    ],
    "ask": [
      "Bash(git push *)"
    ]
  }
}
```

- Save as `.claude/settings.json` and commit it, so every coauthor works under the same rules
- Applies in every mode. Deny rules are checked first and hold even when permission checks are skipped
- **Limits:** rules match tool calls and the literal command text. `/bin/rm -rf` or `bash -c` gets past them, and `grep -r` or a script can still read a denied file
- Worth setting up, but it is not a sandbox

Requests Versus Enforcement

Mechanism	Where it lives	What it does	Kind
CLAUDE.md	project root	standing context, every session	asks
Path-scoped rule	`.claude/rules/*.md`	loads when matching files are touched	asks
Skill	`.claude/skills/<name>/`	a procedure, loaded when relevant	asks
Sub-agent	`.claude/agents/<name>.md`	a separate context for a delegated job	asks
Permission rule	`settings.json`	allows, asks about, or refuses tool calls	**enforces**
Hook	`settings.json`	your script at fixed points; can block	**enforces**
Built-in sandbox	`/sandbox`	OS boundary for shell commands	**enforces**
Container / sandbox	outside Claude Code	an operating-system boundary	**enforces**

If something must happen every time, use the bottom half. “Never modify raw data” in CLAUDE.md is a request the model usually follows. `Edit(./data/raw/**)` in the deny list is enforced.

A Hook That Blocks Secrets

```
#!/bin/bash
# .claude/hooks/no-secrets.sh -- a PreToolUse hook on Bash; chmod +x it
INPUT=$(cat)                                    # the proposed command, as JSON
echo "$INPUT" | grep -q "git commit" || exit 0
[ -n "$WRDS_PASSWORD" ] || . ~/.wrds-mcp.env 2>/dev/null
[ -n "$WRDS_PASSWORD" ] || exit 0               # no password to look for
if { { git diff --cached; git diff; } | grep '^+'
     git ls-files -o --exclude-standard -z | xargs -0 cat
   } 2>/dev/null | grep -qF -- "$WRDS_PASSWORD"; then
  echo "Blocked: this commit would include your WRDS password" >&2
  exit 2
fi
exit 0
```

- It looks for the password itself in every change and new file, so `git add -A && git commit` is caught. **Exit 2 blocks**; any other error lets the commit through
- It only checks commits *Claude* makes. Commits you type in your own terminal skip it, so you still need the audit

Credential Hygiene: the Audit

Run from the root of any project before calling it clean:

```
# 1. Was a credential file ever committed?
git log --all --full-history --oneline -- '*.env' '*.env.*' '.mcp.json'
# 2. Does your actual password appear in any commit, or any uncommitted file?
source ~/.wrds-mcp.env; : "${WRDS_PASSWORD:?source your env file first}"
git log --all --oneline -S"$WRDS_PASSWORD"
git grep -l -F --untracked -e "$WRDS_PASSWORD"
```

- All should return nothing. They print commit IDs and file names, never the password
- Search for the password, not the name `WRDS_PASSWORD`: code that reads it from the environment is correct
- If anything turns up: **change the password or key first**, then remove it from the git history

Paper Trails: a Map, Not a Verdict

- Requested up front: `DECISIONS.md`, `LOG.md`, sample-attrition table
- An LLM will write a `DECISIONS.md` entry saying work was done when it was not. The entry is confident and specific, and no commit matches it
- Check each specific claim against the commit history before you rely on it

The line to remember

Documentation tells you where to look. It does not prove the work was done.

No Hand-Typed Numbers

- A pipeline script computes a result, writes it to `results.tex`
- The paper's `.tex` never types the number. It pulls it in with `\input{results/table2_main.tex}`
- Rerun the pipeline and the paper updates at the next compile. Nobody copies a number by hand
- Without this rule, an LLM drafting a results paragraph will write in a plausible number that nothing computed

Demo 4: Git as the Operating System

- Five commands and one habit. The handout has the commands; this is the demo
- Live: `git diff HEAD~1` after a real Claude session shows a 12-line diff you can actually review
- One rollback, live
- **Review one commit at a time.** At the start of a project, tell Claude to commit at every meaningful step

The Reviewable Diff

capstone — zsh

```
$ git log -1 --oneline
3f9a1c2 Keep firm-years with unused NaNs: dropna on at, sale

$ git diff HEAD~1 --stat
 clean_fsds.py | 12 +++++-------
 1 file changed, 5 insertions(+), 7 deletions(-)

$ git diff HEAD~1 -- clean_fsds.py
-df = df.dropna()
-df['assets'] = df['at']
+df = df.dropna(subset=['at', 'sale'])
+df['assets'] = df['at'].astype(float)
```

Stylized session transcript (not a screenshot).

A commit is the unit of verification: a 12-line diff you can actually review.

A 12-line commit with a message that says why the change was made. This is the size of change you can check properly.

What to Set Up After This Course

1. **Every project, day one:** CLAUDE.md (data, conventions, verification rules), a progress log, git
2. **When a correction repeats:** the third time, make it a skill. Permission rules once the project holds credentials or licensed data
3. **When a pattern repeats across projects:** sub-agents, hooks, global skills

Build each piece when a real repetition or risk calls for it, not in advance. Autonomy is one dial; configuration is another.

Echoes the three-tier adoption structure in Spina's Claude Code brownbag.

Into the Capstone

- Choose a Module 3 (EDGAR panel) or Module 4 (WRDS/fallback funda) pipeline
- Harden it against the rubric: reproducibility, verification, hygiene, environment (stretch)
- Before you start, take 30 seconds to write down one task you will always delegate and one you will never let an agent start

Capstone Rubric

- **Reproducibility** — clean-clone rerun from the README alone; meaningful commits at every checkpoint
- **Verification** — DECISIONS.md/LOG.md/attrition cross-checked against commits; two sourced sanity checks; one number traced raw data → `results.tex` → output, zero hand-typing
- **Hygiene** — both audit checks run, clean, and pasted; `.gitignore` correct; tunnel down; one banal artifact delegated, one meaningful artifact written by hand

Stretch: rerun inside a sandbox and paste the blocked-access output. You grade yourself, then check one `DECISIONS.md` claim against its commit cold. With a group, I spot-check 3–4 projects live.

Module 6: Readings

- Required (2025–26): Bastani et al.; Budzyń et al.; Brodeur et al.; Lu et al.; Kusumegi et al.; Hao, Xu, Li & Evans
- Accounting option: Lu, Luo & Yu (2026); de Kok (2025); or Eisfeldt & Schubert (2025)
- About 90 minutes; read them before you start Module 6. Memo prompt: “What I will still be paid for in 2036”
- Debate positions: assigned when run with a group. Alone, argue the side you find least convincing

Self-Check and Exit Ticket

- Check one DECISIONS.md claim against the commit log, cold. With a group, we do this on screen for 3–4 projects
- Write down your delegation rule: one task you always delegate, one you never do. Then ask what a coauthor would need to see before trusting your repository
- Exit ticket: the standard three questions, plus your delegation rule in writing

The Course Operating System

The Course Operating System

Five commands, one habit, three mantras, one honest paper trail — everything Module 5 asks you to keep doing after the course ends.

1.	2.	3.
Files persist; context doesn't.	**Trust, but verify — same as an RA.**	**Be specific: file, operation, aggregation, output.**

The Five Git Commands

`git init`	Turn a folder into a repository. Run once, before anything else happens in the project.
`git add -A && git commit -m "msg"`	The checkpoint. Snapshot everything now, message says why, not just what. Commit at every real milestone.
`git log --oneline`	The history. A short, readable list of every checkpoint in order.
`git diff HEAD~1`	The review unit. One bounded, readable diff — never 800 lines at once.
`git push` / `git pull`	Sync with a remote. Push after committing; pull before starting a shared project.

Paper-Trail Files

`DECISIONS.md`	Every methodological choice, recorded as it's made — which tag, how duplicates were handled, what filter and why.
`LOG.md`	A timestamped line for every major step of the pipeline.
Attrition row	n before, n after, reason — printed every time the sample size changes.
`results.tex` `\input{}`	Pipeline writes numbers to `results.tex`; the paper `\input{}`s it. Zero hand-typed numbers, anywhere.

Credential Audit — Two Checks (run both, expect nothing back)

1. Was a credential file ever committed, in any folder?

```
git log --all --full-history --oneline -- '*.env' '*.env.*' '.mcp.json'
```

Expected: an empty result. A hit means a credential is exposed in history even if the file is deleted now.

2. Does your actual password appear in any commit, or in any uncommitted file?

```
source ~/.wrds-mcp.env; : "${WRDS_PASSWORD:?source your env file first}"
git log --all --oneline -S"$WRDS_PASSWORD"
git grep -l -F --untracked -e "$WRDS_PASSWORD"
```

Expected: nothing. Search for the password itself, not its name. Repeat for every other secret your project uses.

Hygiene Checklist

- ☐ `.gitignore` excludes `.env`, raw WRDS-derived data, and any other credential or licensed-data file.
- ☐ Both credential-audit checks above re-run clean — pasted output, not "it's clean," is the deliverable.
- ☐ Iron-law filters documented for every WRDS pull this project made (Module 4 card).
- ☐ WRDS tunnel closed (`tunnel_down.sh` / exit the session) before you walk away.